

Claude Code 复杂任务长久稳定运行机制分析报告

> 阅读提示：本文假设读者有一定编程基础，但不需要了解 Claude Code 内部实现。每个概念都会先解释"为什么需要它"，再说明"它是怎么工作的"。

>

> 分析日期：2026-07-07 | 项目版本：Claude Code 源码快照 (约 1,900 个 TypeScript 文件, 约 512,000 行代码)

目录

- 整体架构概览
- 任务生命周期管理
- API 重试引擎
- 上下文窗口管理
- Agent 编排与子任务管理
- 错误处理与恢复机制
- 优雅关闭与进程级保障
- 后台清理与磁盘管理
- 可靠性模式总结
- 关键文件索引

1. 整体架构概览

1.1 这个项目在做什么？

Claude Code 是一个终端里的 AI 编程助手。你可以在命令行里跟它对话，让它帮你写代码、查资料、运行命令。它会复杂任务拆分成多个子任务，交给不同的"助手" (Agent) 去并行执行。

1.2 为什么需要稳定性机制？

想象一下这个场景：

> 你让 Claude Code "重构整个项目的认证模块"。这个任务可能需要：

- 先阅读 20 个相关文件
- 理解现有的认证流程
- 设计新的架构
- 修改 10 多个文件
- 运行测试验证

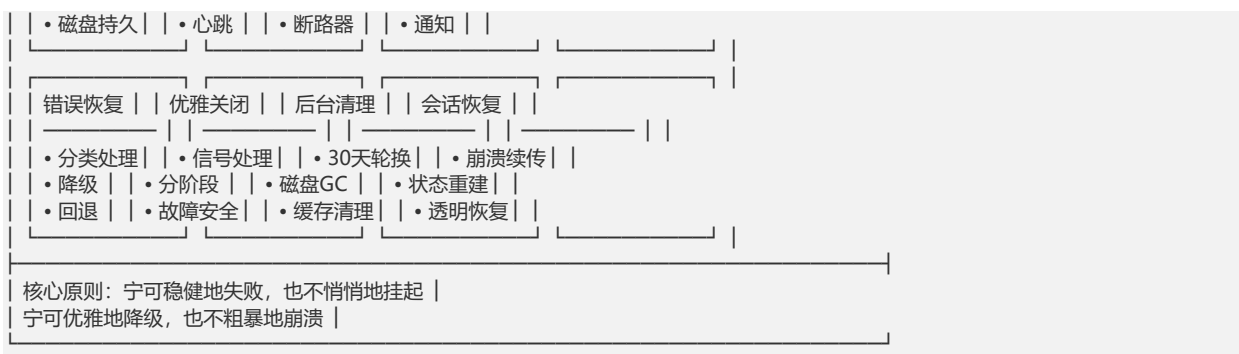
>

> 整个过程可能持续几分钟甚至几十分钟。期间可能遇到：API 临时故障、网络波动、上下文窗口满了、用户不小心按了 Ctrl+C、电脑要休眠了.....任何一个环节出问题，前面的工作就白费了。

所以，Claude Code 需要一套能够应对各种意外情况的可靠性机制。

1.3 整体设计哲学





没有"万能检查点"：Claude Code 并没有一个单一的"保存进度"按钮。它的可靠性来自于多个子系统各司其职、层层兜底：

- 每条消息都存盘 → 崩溃了可以从头恢复对话
- API 调用有重试 → 网络波动不会导致失败
- 上下文窗口有预警 → 不会突然"失忆"
- 子任务有隔离 → 一个助手崩溃不影响其他助手
- 关闭时有清理 → 不会留下垃圾进程或损坏的文件

2. 任务生命周期管理

2.1 为什么需要任务管理？

当 Claude Code 执行一个复杂任务时，它可能需要同时运行：

- 几个 shell 命令（比如编译、测试）
- 几个子助手（比如一个读文件，一个查资料）
- 一个 MCP 服务监控器

这些任务有的快的慢，有的成功有的失败。系统需要：

1. 跟踪每个任务的状态（开始了吗？结束了吗？）
2. 收集每个任务的输出（结果是什么？）
3. 在任务完成后清理资源（别占着内存不放）

2.2 统一任务抽象

所有任务都遵循同一个生命周期状态机：



为什么需要状态机？状态机让系统能明确知道"每个任务现在处于什么阶段"，避免出现"任务已经结束了但还在给它发消息"或"任务还在运行但资源被回收了"这类问题。

七种任务类型：

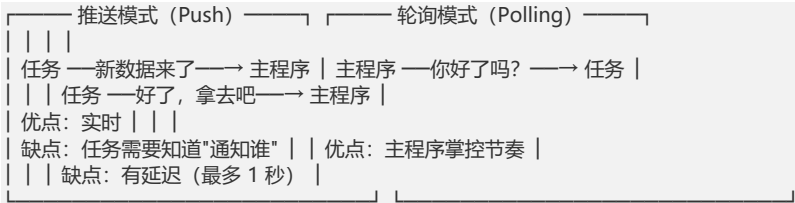
任务类型	图标	说明	例子
`local_bash`	终端	本地 shell 命令	`npm install`、`git status`
`local_agent`	助手	本地子助手	让一个子助手去读文件
`remote_agent`	远程	远程助手（Bridge 模式）	通过 IDE 插件执行
`in_process_teammate`	队友	同进程队友	和另一个 AI 协作
`local_workflow`	工作流	工作流脚本	预设的自动化流程
`monitor_mcp`	监控	MCP 服务器监控	监控外部服务状态
`dream`	梦境	后台推测	预判用户下一步操作

安全设计：每个任务都有一个加密随机 ID ($36^8 \approx 2.8$ 万亿种可能)，防止恶意程序伪造任务 ID 来读取任务输出。

2.3 为什么需要检查任务状态？

任务在执行时，会持续产生输出（比如命令的 stdout、子助手的中间结果）。这些输出存在磁盘上，但主程序需要定期查看是否有新内容。

事件驱动 vs 定时轮询：

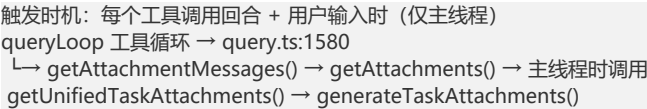


Claude Code 选择事件驱动模式，原因如下：

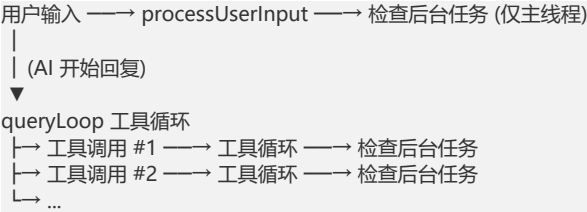
- 任务可能是在隔离环境中运行的，无法直接通知主程序
- 在工具循环的每次迭代中顺带检查，实现简单可靠（不需要维护定时器或长连接）
- 每次工具循环迭代之间最多 1 秒的延迟，对大多数任务来说可以接受
- 注意：pollTasks() 函数虽然定义了（含 POLLINTERVALMS = 1000 常量），但实际没有被任何代码调用——真正的检查是通过 generateTaskAttachments() 在工具循环和用户输入处理时触发的

2.4 任务状态检查机制

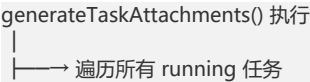
重要纠正：pollTasks() 函数虽然定义了（src/utils/task/framework.ts:255），但从未被任何代码调用。真正被调用的是 generateTaskAttachments()，它通过事件驱动的方式触发：



任务状态检查不是定时轮询，而是事件驱动的——每次工具循环迭代或用户输入时顺带检查一次：



pollTasks() 中封装的实际逻辑与 generateTaskAttachments() 一致（读取磁盘增量、生成偏移量补丁、通知），只是入口不同。但 pollTasks() 本身是死代码——如果有人调用它，它的行为是：





关键概念解释：

什么是"乐观驱逐"？

- 普通做法：等任务自己说"我清理完了"再删除
- 乐观做法：先标记为"可以删了"，再清理。如果清理出问题，重新创建即可
- 好处：不会因为清理失败而阻塞整个系统

为什么需要 30 秒宽限期？

- 协调器面板（Coordinator Panel）会显示所有子助手的状态
- 如果任务刚完成就立刻删除，用户可能来不及看到"任务已完成"的提示
- 30 秒宽限期让用户能看清结果，再优雅地消失

为什么用"偏移量补丁"而不是全量状态？

- 全量状态更新：A 线程正在写状态，B 线程也在写，可能互相覆盖（zombification bug）
- 偏移量补丁：只传"从第 N 行到第 M 行的新内容"，互不干扰
- 类似 git 的增量 diff，而不是每次提交整个文件

2.5 磁盘输出持久化

每个任务的输出都会写入磁盘文件：



为什么是 5GB 上限？

- 防止某个任务输出无限增长（比如误写了死循环）
- 同时也防止其他任务无法写入（磁盘空间被占满）
- 达到上限时，会终止进程并显示清晰的截断消息

为什么需要安全打开标志？

- O_NOFOLLOW：如果攻击者把输出文件替换成指向敏感文件的符号链接，不加这个标志就会写入敏感文件
- O_EXCL：防止多个任务同时写入同一个文件导致数据错乱

3. API 重试引擎

3.1 为什么需要重试？

Claude Code 依赖 Anthropic 的 API 来调用 AI 模型。API 调用可能因为各种原因失败：

常见的 API 失败原因：

	529	—	服务器过载 ("我们太忙了，稍后再试")	
	429	—	请求过于频繁 ("你太快了，慢一点")	
	5xx	—	服务器内部错误 ("我们出错了")	
	408	—	请求超时 ("网络太慢了")	
	ECONNRESET	—	连接被重置 ("网络断了")	
	401	—	认证过期 ("登录信息失效了")	

如果遇到失败就直接放弃，用户会得到一个"对不起，我失败了"的回复，体验很差。所以需要自动重试。

3.2 重试策略：指数退避

尝试次数 等待时间 累计等待

第1次	立即重试	0s
第2次	等待 ~1 秒	1s
第3次	等待 ~2 秒	3s
第4次	等待 ~4 秒	7s
第5次	等待 ~8 秒	15s
第6次	等待 ~16 秒	31s
第7次	等待 ~32 秒	63s
第8次	等待 ~32 秒 (上限)	95s
第9次	等待 ~32 秒 (上限)	127s
第10次	等待 ~32 秒 (上限)	159s

公式：等待时间 = BASE_DELAY × 2^(尝试次数-1)，上限 32 秒，再加上 25% 的随机抖动

为什么需要指数退避？

- 如果每次重试都等同样长的时间，服务器可能还在过载，重试也是白费
- 指数增长让重试间隔迅速扩大，给服务器恢复的时间
- 随机抖动 (jitter) 防止多个客户端同时重试 ("惊群效应")

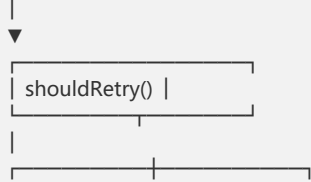
为什么需要上限？

- 如果没有上限，第 20 次重试要等 2^19 ≈ 6 天，这显然不合理
- 32 秒是个平衡点：既给了足够的时间恢复，又不会让用户等太久

3.3 精细化错误分类

不是所有错误都值得重试。系统会区分对待：

API 返回错误



重试 放弃

	529 (过载)		400 Bad Request
	↳ 最多3次，之后		(请求本身有问题，重试也没用)
	切换到备用模型※		
	404 Not Found		
	429 (限流)		(资源不存在)
	↳ 非订阅用户或		
	Enterprise 重试		



> ※ 529 切换到备用模型有条件：仅当设置了 `FALLBACKFORALLPRIMARYMODELS` 环境变量，或者用户不是 Claude AI 订阅者且使用的是非自定义 Opus 模型时才会触发模型回退。订阅用户（非 Enterprise）遇到 529 不会触发自动回退。

3.4 持久无人值守模式

对于长时间运行的后台任务（比如通宵重构代码），有专门的持久重试模式：

普通模式：持久模式：
最多重试 10 次 无限重试（直到成功或用户取消）
最大退避 32 秒 最大退避 5 分钟
没有心跳 每 30 秒发送一次心跳

为什么需要心跳？

- 如果 API 在长达几分钟的退避等待中没有响应，宿主环境（比如 IDE 插件）可能会认为会话已经“死”了，从而关闭连接
- 每 30 秒发送一个 `api_retry` 系统消息，告诉宿主：“我还活着，只是在等 API 恢复”

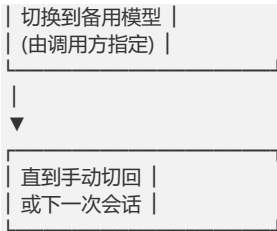
为什么持久模式退避上限是 5 分钟？

- 对于过载类错误（529），API 通常几分钟内就能恢复
- 5 分钟是一个合理的平衡点：既不会太频繁地打扰 API，也不会让用户等太久

3.5 模型回退

连续 3 次 529 错误





- 为什么需要模型回退？
- 主模型可能因为过载而暂时不可用
 - 备用模型通常更小、更快、更便宜
 - 与其让用户干等，不如降级使用一个稍差的模型继续工作

- 为什么是 3 次？
- 1 次可能是偶然波动
 - 2 次可能是不巧
 - 3 次基本可以确定是持续性问题，值得切换了

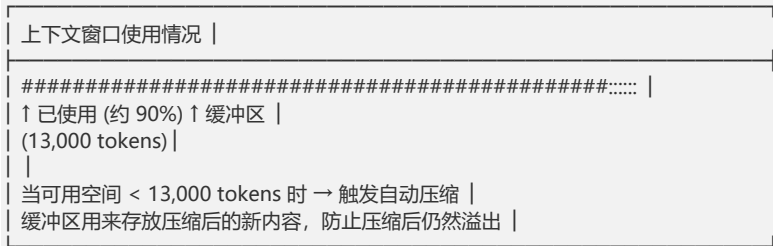
- 谁可以触发回退？
- 仅当设置了 FALLBACKFORALLPRIMARYMODELS 环境变量（覆盖所有场景），或用户不是 Claude AI 订阅者且使用的是非自定义 Opus 模型时
 - 这意味着大多数订阅用户（使用 Sonnet 等非 Opus 模型）遇到 529 时不会触发模型回退，而是继续重试

4. 上下文窗口管理

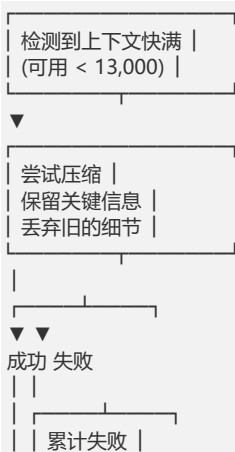
4.1 为什么需要上下文管理？

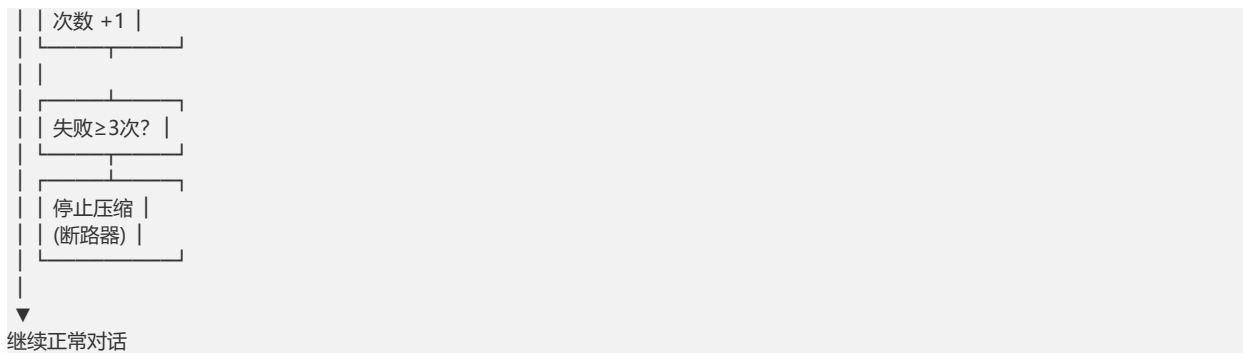
AI 模型有一个"上下文窗口"——它能记住的对话内容上限。Claude 的上下文窗口很大（100K-200K tokens），但在长时间的对话中仍然可能被填满。

想象一下：你让 AI 帮你重构了 50 个文件，每个文件的修改都要在上下文里保留。当上下文满了，AI 会"忘记"最早的文件内容，导致工作出现不一致。



4.2 自动压缩流程





为什么需要缓冲区 (13,000 tokens) ?

- 压缩需要时间来执行，期间对话还在继续
- 缓冲区确保压缩过程中不会出现"溢出"的尴尬情况
- 13,000 tokens 约等于 10,000 个英文单词，足够压缩期间使用

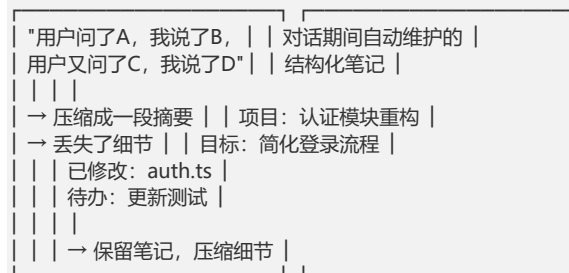
为什么需要断路器 (连续 3 次失败后停止) ?

- 如果压缩连续失败，说明压缩本身有问题 (比如格式不对)
- 继续尝试只会浪费 API 调用次数和用户的钱
- 数据显示每天约 25 万次压缩调用，断路器能节省大量无效调用

4.3 会话内存压缩

除了传统的压缩方式，还有一种更智能的"会话内存"压缩：

传统压缩： 会话内存压缩：



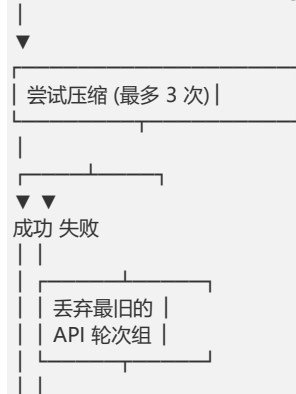
会话内存的好处：

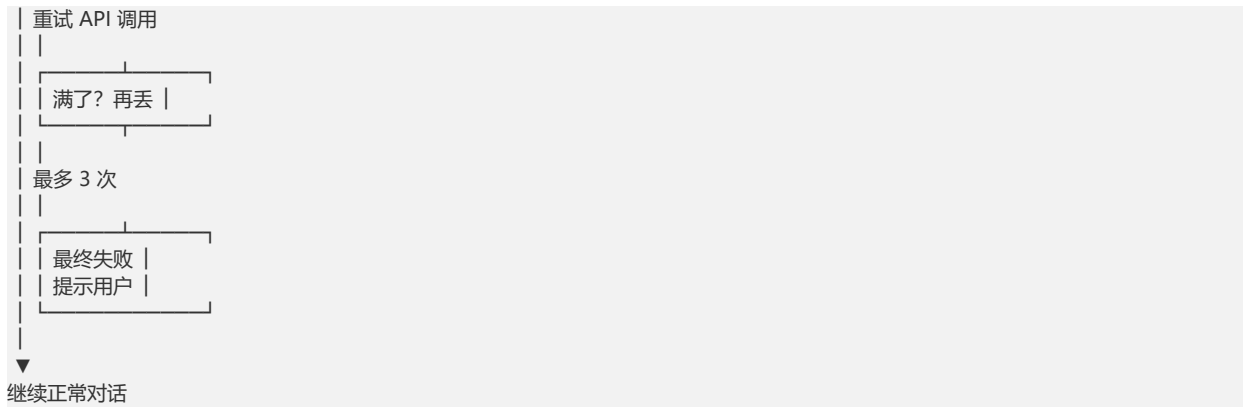
- 由后台助手自动维护，不需要用户手动操作
- 内容是结构化的，保留关键信息
- 压缩时保留笔记，只丢弃无关细节
- 如果会话内存不可用，优雅回退到传统压缩

4.4 PTL (Prompt Too Long) 重试

如果不幸真的遇到了"提示太长"错误，还有专门的恢复机制：

API 返回 400 "prompt too long"





5. Agent 编排与子任务管理

5.1 为什么需要 Agent 编排?

当任务复杂时，一个 AI 助手可能忙不过来。比如：

> 用户："帮我分析这个项目的性能瓶颈"

>

> 如果只有一个助手，需要：

> 1. 读所有源码 → 耗时

> 2. 运行性能测试 → 不能同时做

> 3. 分析结果 → 要等前两步

>

> 如果有多个助手并行：

> 助手A：读前端代码

> 助手B：读后端代码

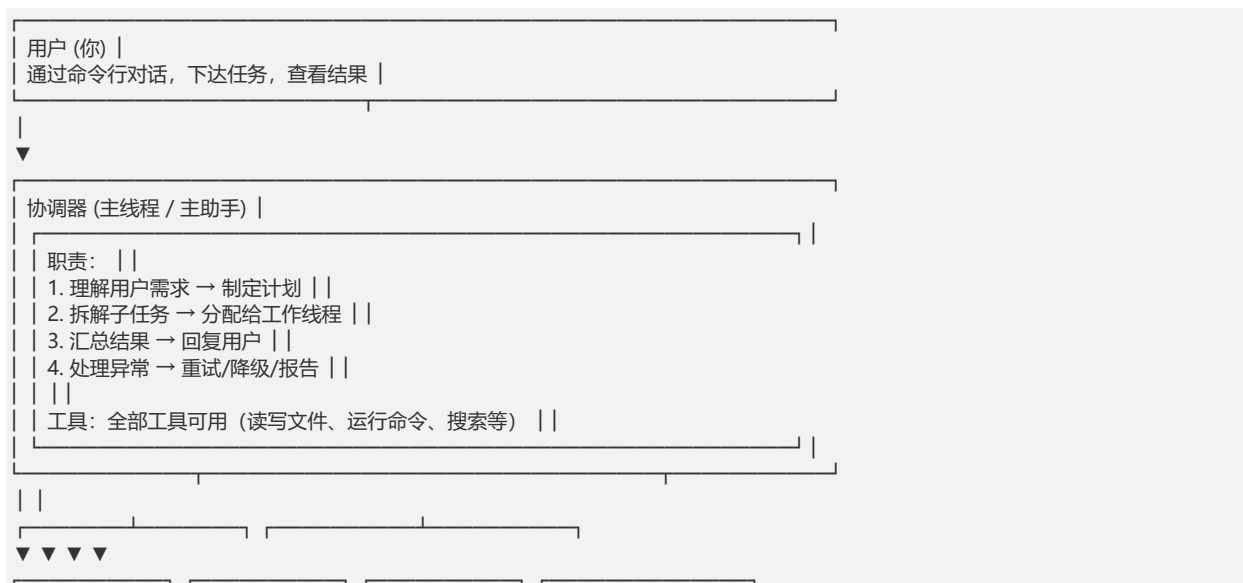
> 助手C：运行性能测试

> 助手D：检查数据库查询

>

> 然后主助手汇总所有结果 → 快得多！

5.2 三层架构



工作线程A	工作线程B	工作线程C	...	工作线程N
(读文件)	(查资料)	(运行测试)	(写代码)	
工具受限	工具受限	工具受限	工具受限	
只读文件	只读文件	只运行命令	读写文件	
+ 搜索				
独立状态	独立状态	独立状态	独立状态	

- 为什么协调器和工作线程要分开？
- 模块化：协调器专注于"计划"，工作线程专注于"执行"
 - 容错：一个工作线程崩溃，不影响其他工作线程和协调器
 - 并行：多个工作线程可以同时工作，大幅提高效率
 - 隔离：工作线程的权限通常更受限（比如只读文件），降低安全风险

5.3 四阶段工作方法

协调器按照以下流程组织工作（实际代码中定义了 4 个阶段）：

阶段1: 研究 (Research)
并行启动多个工作线程，各自探索不同的方向
工作线程A: 读项目结构
工作线程B: 搜索相关代码
工作线程C: 查阅文档
协调器: 等待所有结果返回
▼
阶段2: 综合 (Synthesis)
协调器自己分析所有工作线程的结果
阅读研究发现，理解问题，制定实现规格说明
▼
阶段3: 实现 (Implementation)
启动新的工作线程去执行具体修改
工作线程D: 修改文件A
工作线程E: 修改文件B
工作线程F: 修改文件C
▼
阶段4: 验证 (Verification)
启动工作线程验证修改是否正确
工作线程G: 运行测试
工作线程H: 检查代码风格

完成后，协调器直接向用户报告结果（报告是协作流程的固有部分，并非独立阶段）。

5.4 Forked Agent 隔离机制

当工作线程启动时，它不是随便运行的——它在一个隔离的环境中执行：

父进程 (主助手)
AbortController (主控制器)
readFileState (已读文件列表)
toolDecisions (工具使用决策)
systemPrompt (系统提示词)
createChildAbortController()
createSubagentContext()
▼
子进程 (工作线程)

```
| AbortController (子控制器) ← 链接到父级 |
| readFileState (克隆副本) ← 修改不影响父级 |
| toolDecisions (全新空列表) ← 与父级隔离 |
| systemPrompt (共享引用) ← 只读, 安全共享 |
| shouldAvoidPermissionPrompts = true |
| ↳ 后台任务不弹出权限对话框 |
```

为什么需要隔离？

- 防止污染：工作线程读文件时出错，不应该影响主助手的状态
- 防止阻塞：工作线程卡住了，主助手还能继续工作
- 防止权限冲突：工作线程在后台运行，不应该弹出对话框让用户确认

子 AbortController 的妙用：

- 父级中止 → 子级自动中止（级联取消）
- 子级中止 → 不影响父级（单个任务失败不影响整体）
- 使用 WeakRef 防止内存泄漏（不会因为忘记清理引用导致内存占用）

5.5 工作线程的完整生命周期

创建

```
▼
| AgentTool 被调用 |
| 分配 agentId (如 "a3f8b2c1") |
| 确定 subagent_type |
```

注册

```
▼
| registerAsyncAgent() |
| 添加到 AppState.tasks |
| 分配 AbortController |
| 状态: pending → running |
```

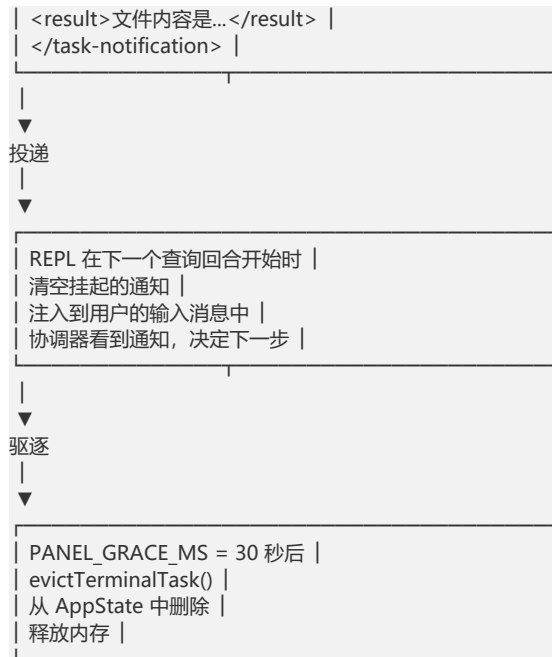
执行

```
▼
| runAgent() |
| 构建隔离上下文 |
| 调用 query() → LLM 循环 |
| 每 30 秒生成一次进度摘要 |
| 输出写入磁盘 |
```

成功 失败

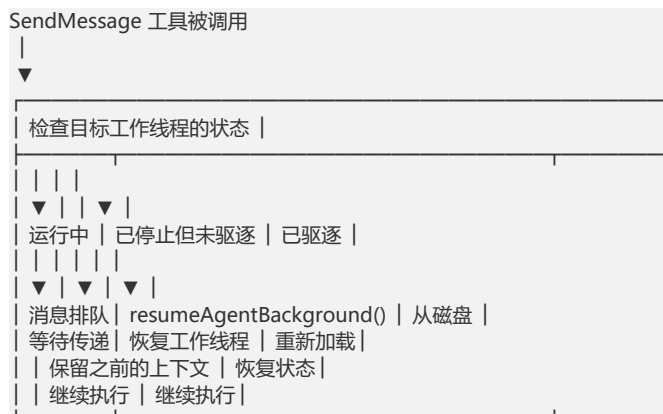
```
▼ ▼
| |
| completeAgentTask | |
| 发送完成通知 | |
```

```
▼ ▼
| enqueueAgentNotification() |
| 发送 XML 格式的通知到消息队列 |
| <task-notification> |
| <task-id>a3f8b2c1</task-id> |
| <status>completed</status> |
| <summary>完成了文件读取</summary> |
```



5.6 继续已停止的工作线程

如果工作线程完成了第一阶段的工作，但后续还需要它怎么办？



5.7 顺序执行保证

有时候多个后台任务需要按顺序执行，比如：

后台任务：会话内存提取（每 30 秒一次）
提示建议生成（用户输入时触发）

如果没有顺序保证：

内存提取 → 还没完成 → 提示建议又来了
→ 两个任务同时修改同一个文件 → 数据错乱

有了顺序保证（sequential.ts）：

内存提取 → 排队 → 提示建议 → 等待
→ 内存提取完成 → 提示建议开始
→ 顺序执行，互不干扰

6. 错误处理与恢复机制

6.1 为什么需要分层错误处理？

不同的错误需要不同的处理方式：

错误类型	严重程度	处理方式
用户按了 Ctrl+C	低	静默忽略，不记录日志
命令执行失败（如 `rm` 不存在的文件）	低	记录 debug 日志，继续执行
磁盘空间不足	中	尝试清理，提示用户
API 认证过期	高	刷新令牌，重试请求
代码崩溃	严重	记录诊断信息，优雅退出

分层处理的好处：

- 不把用户当专家：普通错误给简单提示，复杂错误给详细信息
- 不浪费日志空间：低级别错误不记录到 error 日志
- 不掩盖问题：严重错误一定会被注意到

6.2 25+ 错误场景分类

API 错误分类树

classifyAPIError() 返回的错误类型 |

├─ aborted 用户取消/超时 |

├─ api_timeout API 响应超时 |

├─ repeated_529 连续过载错误 |

├─ capacity_off_switch 容量开关关闭 |

├─ rate_limit 请求速率限制 |

├─ server_overload 服务器过载 |

├─ prompt_too_long 提示过长 |

├─ pdf_too_large PDF 文件过大 |

├─ pdf_password_protected PDF 有密码保护 |

├─ image_too_large 图片文件过大 |

├─ tool_use_mismatch 工具使用不匹配 |

├─ auth_error 认证失败 |

├─ invalid_api_key API 密钥无效 |

├─ token_revoked 令牌已撤销 |

├─ credit_balance_low 额度不足 |

├─ invalid_model 模型不可用 |

├─ server_error 服务器内部错误 |

├─ connection_error 网络连接错误 |

├─ ssl_cert_error SSL 证书错误 |

└─ unknown 未知错误 |

6.3 工具执行时的错误处理

当 AI 调用某个工具（比如读文件、写文件）时，工具可能出错：

工具执行出错

|

▼

AbortError? |

(用户取消了操作) |

└─ 静默忽略，不记录日志 |

不打扰用户 |

ShellError? |

(命令执行出错) |

└─ 记录到 debug 日志 |

错误信息会返回给 AI 处理 |

其他错误? |

└─ 记录到 debug + error 日志 |

错误信息会返回给 AI 处理 |

所有错误最终: |

```
| ↳ formatError() → tool_result |  
| 格式化 AI 能理解的消息格式 |  
| AI 自己决定下一步做什么 |
```

关键设计：工具错误不会导致程序崩溃。错误被封装成消息返回给 AI，由 AI 决定是重试还是放弃。这就像"上级把问题抛回给当事人自己解决"。

6.4 对话恢复（崩溃后续传）

如果程序崩溃了，用户重新启动时，需要能接着之前的地方继续：

崩溃前：

用户：帮我重构 auth 模块

AI：好的，我先读一下 auth.ts

AI：[读文件 auth.ts] ← 读到一半，崩溃了！

AI：[读文件 auth.ts] ← 读到一半，崩溃了！

AI：[读文件 db.ts] ← 输出被截断

崩溃后恢复：

```
| deserializeMessagesWithInterruptDetection() |  
| |  
| 检测到：流被中断了！ |  
| 恢复：插入一条消息 |  
| "Continue from where you left off" |  
| |  
| filterUnresolvedToolUses() |  
| 移除：孤立的 tool_use 块 |  
| （中断时写到一半的请求，删掉） |  
| |  
| filterOrphanedThinkingOnlyMessages() |  
| 移除：只有思考过程没有结果的消息 |  
| |  
| filterWhitespaceOnlyAssistantMessages() |  
| 移除：空白的助手消息 |  
| （中断时可能产生了空消息） |  
| |  
| restoreSkillStateFromMessages() |  
| 恢复：跨压缩周期保留技能状态 |
```

为什么需要这么多过滤步骤？

- 流式输出可能在任何位置被截断
- 不完整的消息（比如只有 tooluse 没有 toolresult）会导致 API 错误
- 这些过滤步骤确保恢复后的消息是"干净"的，不会触发新的错误

6.5 会话状态恢复

当用户用 --resume 参数恢复会话时，系统会重建所有状态：

启动恢复

```
|  
| ▼  
| 从 JSONL 日志读取所有消息 |  
| （每条消息都有时间戳和顺序号） |
```

```
|  
| ▼ ▼  
| 恢复文件历史 | | 恢复属性状态 |  
| 知道 AI 读过哪些 | | 知道哪些代码是 |  
| 文件 | | AI 写的 |
```

```
|  
| ▼ ▼  
| 恢复待办事项 | | 恢复 Agent 设置 |
```



为什么恢复这么复杂？

- 因为 Claude Code 是一个有状态的应用，不是简单的"一问一答"
- 它知道读过哪些文件、写过哪些代码、还有哪些待办事项
- 如果这些状态丢失，用户就要重新告诉 AI 所有上下文

7. 优雅关闭与进程级保障

7.1 为什么需要优雅关闭？

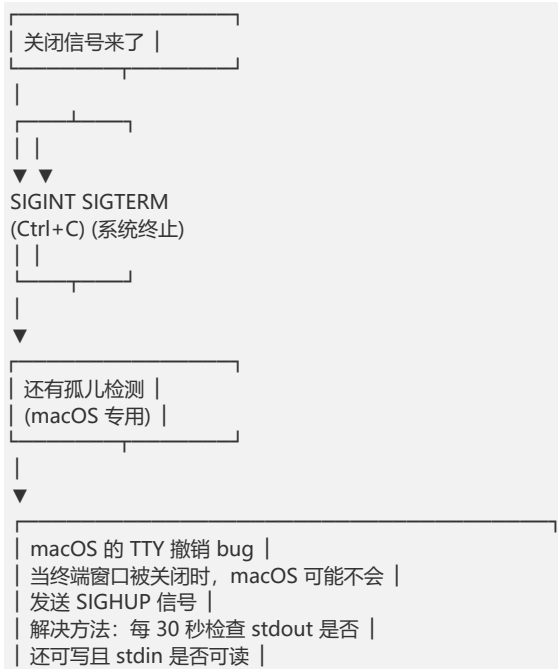
当用户关闭程序时，不能直接"拔电源"：

[X] 粗暴关闭：
用户按 Ctrl+C → 进程立刻终止
→ 正在写的文件可能损坏
→ 正在运行的子进程变成孤儿
→ 终端显示混乱（没有恢复鼠标模式）
→ 下次恢复时找不到日志文件

[OK] 优雅关闭：
用户按 Ctrl+C → 开始清理流程
→ 通知子进程结束
→ 等待正在写的文件写完
→ 恢复终端设置
→ 打印恢复提示
→ 然后退出

7.2 信号处理

Claude Code 处理三种关闭信号：



| 如果任何一项不可用 => 终端已死 => 关闭 |

7.3 分阶段关闭流程

关闭信号
▼
第1步：启动故障安全计时器 └─ 最多等 max(5s, 钩子超时 + 3.5s) └─ 时间一到，不管清理是否完成，强制退出 └─ 防止清理过程本身卡死
第2步：终端清理 └─ 关闭鼠标追踪 └─ 退出 alt 屏幕 └─ 恢复键盘模式 └─ 显示光标 └─ 防止终端显示混乱
第3步：打印恢复提示 └─ "claude --resume abc123" └─ 用户下次可以用这个命令恢复对话
第4步：运行清理函数 (2s 超时) └─ 关闭打开的文件 └─ 清理临时文件 └─ 终止子进程
第5步：执行 SessionEnd 钩子 └─ 运行用户配置的关闭脚本 └─ AbortSignal.timeout() 防止卡死
第6步：刷新分析数据 (500ms 上限) └─ 把未发送的分析数据发出去
第7步：强制退出 └─ process.exit() └─ 如果抛出 EIO (终端已死) └─ 回退到 SIGKILL

7.4 防止睡眠

对于长时间运行的任务，Mac 电脑可能会自动休眠。Claude Code 会阻止这种情况：

preventSleep.ts
机制：
caffeineate 进程 (Mac 自带)
每 5 分钟超时自动重启
每 4 分钟检查一次，防止超时
引用计数：
startPreventSleep() → 计数 +1
stopPreventSleep() → 计数 -1
计数 > 0 时保持唤醒
计数 = 0 时允许睡眠
自动清理：
exit 时通过 cleanupRegistry 终止 caffeineate

为什么需要引用计数？

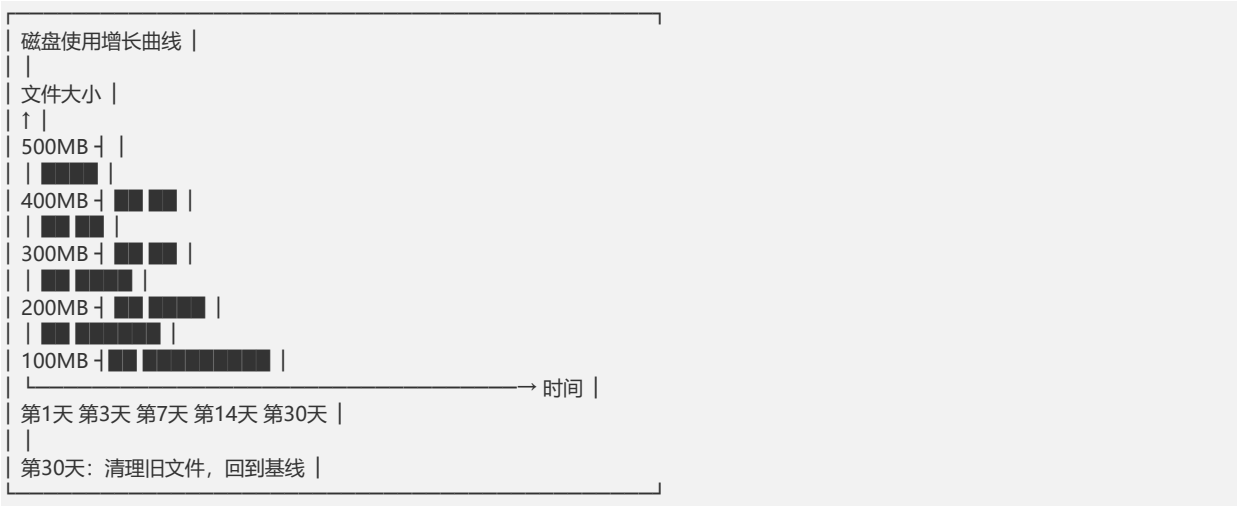
- 多个组件可能同时请求"不要睡眠"

- 如果其中一个组件说"可以睡了"就真的睡了，其他组件还在工作
- 引用计数确保所有组件都完成后才允许睡眠

8. 后台清理与磁盘管理

8.1 为什么需要后台清理？

长时间使用后，Claude Code 会在磁盘上积累大量文件：



8.2 清理内容

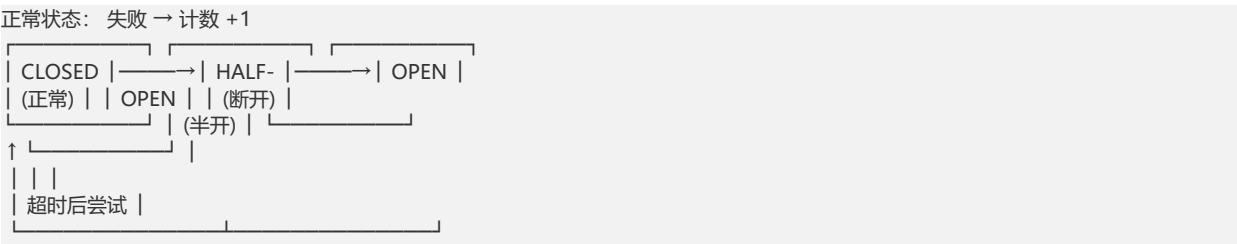
清理项	为什么需要清理	保留期
旧会话文件 (JSONL)	记录了每次对话，时间久了会很大	30 天
旧消息文件 (错误日志)	调试用，但不需要永久保留	30 天
旧计划文件	之前生成的计划文档	30 天
旧代理工作区 (worktree)	Git 分支，不清理会越来越多	30 天
图像缓存	AI 生成的图片缓存	30 天
npm 缓存	`@anthropic-ai/claude-` 包的老版本	每包保留最新 5 个
旧版二进制文件	自动更新的旧版本	标记文件 + 锁控制

30 天原则：大多数调试场景下，1 个月内的日志足够排查问题。超过 1 个月的日志，基本不会有人去翻了。

9. 可靠性模式总结

9.1 电路断路器模式

什么是电路断路器？就像家里的电闸——当电流过大时自动跳闸，防止烧毁电器。在软件中，当某个操作连续失败时，自动停止尝试，防止浪费资源。



(恢复成功，重新闭合) (放弃重试)

Claude Code 中的断路器：

断路器	触发条件	触发后的行为
自动压缩	连续 3 次压缩失败	停止尝试压缩，下次对话再试
模型回退	连续 3 次 529 错误（且满足 gating 条件）	切换到备用模型，持续使用直到手动切换
快速模式冷却	遇到 429/529	快速模式禁用 10 分钟

9.2 回退链模式

什么是回退链？ 当一个方案失败时，按优先级尝试下一个方案，直到成功或所有方案都失败。

主方案 → 方案A → 方案B → 方案C → 报告失败
↑
每个方案失败时自动降级

Claude Code 中的回退链：

场景	回退链
模型调用	主模型 → 备用模型（由调用方指定）
上下文压缩	缓存共享 → 流式压缩 → 截断头部 → 下次自动重试
安全存储	macOS keychain → 纯文本文件（位置更安全）
终端关闭	`process.exit()` → `SIGKILL`（最后手段）
任务输出	符号链接 → 普通文件（符号链接不支持时）
会话压缩	会话内存压缩 → 传统压缩摘要

9.3 TOCTOU 安全模式

什么是 TOCTOU？ Time of Check to Time of Use——检查时刻和使用时刻之间的时间差。在这段时间里，条件可能已经变了。

```
[X] 不安全：
if (文件存在) { ← 检查
// 其他程序可能在这时删除了文件
read(文件) ← 使用（文件已不存在）
}

[OK] 安全：
try {
read(文件) ← 直接使用，不做检查
} catch (e) {
// 处理文件不存在的情况
}
```

Claude Code 中的 TOCTOU 防护：

场景	防护措施
任务状态更新	针对最新 `prev.tasks` 重新检查，而非陈旧快照
Worktree 恢复	使用 `process.chdir()` 作为原子存在性检查
任务注册替换	替换时携带现有状态向前，不丢失已有数据

9.4 竞态防护

什么是竞态？ 多个线程/进程同时操作同一份数据，导致结果不可预测。

[X] 竞态：
线程A：读取 count = 5

```
线程B: 读取 count = 5
线程A: 写入 count = 6 (5+1)
线程B: 写入 count = 6 (5+1, 覆盖了A的结果)
结果: count = 6, 但应该是 7

[OK] 防护:
线程A: 原子操作 count += 1 → count = 6
线程B: 原子操作 count += 1 → count = 7
结果: count = 7
```

Claude Code 中的竞态防护:

机制	说明
任务通知幂等性	原子性检查 + 设置 `notified` 标志, 防止重复通知
顺序执行包装器	`sequential.ts` 确保并发调用逐个执行
任务输出增量补丁	仅偏移量补丁, 避免覆盖状态转换
子 AbortController	`WeakRef` 保护, 防止内存泄漏

10. 关键文件索引

以下是报告中涉及的所有关键源文件, 按功能域分组:

任务管理

文件	内容	行数 (估计)
`src/Task.ts`	任务类型定义、状态机、ID 生成	~200
`src/utils/task/framework.ts`	任务注册、通知驱逐、事件驱动检查	~400
`src/utils/task/diskOutput.ts`	磁盘持久化、5GB 上限、安全文件打开	~300
`src/tasks/stopTask.ts`	类型化任务停止操作	~100
`src/tasks.ts`	任务类型注册表	~50

API 通信

文件	内容	行数 (估计)
`src/services/api/withRetry.ts`	重试引擎、退避算法、回退、心跳	~500
`src/services/api/errors.ts`	25+ 错误场景分类、用户提示	~300
`src/services/api/errorUtils.ts`	连接错误提取、SSL 检测	~150

上下文管理

文件	内容	行数 (估计)
`src/services/compact/autoCompact.ts`	自动压缩、断路器 (3 次失败停)	~200
`src/services/compact/compact.ts`	PTL 重试循环	~300
`src/services/compact/sessionMemoryCompact.ts`	基于会话内存的压缩	~200
`src/services/SessionMemory/sessionMemory.ts`	后台知识提取 (非阻塞)	~300

Agent 编排

文件	内容	行数 (估计)
`src/coordinator/coordinatorMode.ts`	多 agent 编排 (四阶段方法)	~400
`src/utls/forkedAgent.ts`	隔离子 agent 执行	~300
`src/tools/AgentTool/AgentTool.tsx`	核心编排原语	~1900
`src/tools/AgentTool/runAgent.ts`	核心 agent 查询循环	~500
`src/tools/AgentTool/resumeAgent.ts`	从磁盘恢复已停止 agent	~300
`src/tools/SendMessageTool/SendMessageTool.ts`	继续已停止 agent	~400
`src/tools/TaskStopTool/TaskStopTool.ts`	终止 agent	~100
`src/services/AgentSummary/agentSummary.ts`	30 秒进度摘要	~200
`src/utls/messageQueueManager.ts`	通知投递管理	~200
`src/utls/queueProcessor.ts`	命令队列处理	~200

错误处理与恢复

文件	内容	行数 (估计)
`src/utls/sessionRestore.ts`	完整恢复引擎	~550
`src/utls/sessionState.ts`	状态机 (idle/running/requires_action)	~100
`src/utls/conversationRecovery.ts`	中断检测、工具过滤、状态恢复	~300
`src/utls/sessionStorage.ts`	JSONL 持久化	~200
`src/utls/errorLogSink.ts`	缓冲 JSONL 写入器 (1s 刷新)	~150
`src/utls/errors.ts`	自定义错误类	~100
`src/services/tools/toolExecution.ts`	工具执行错误处理	~200

进程级保障

文件	内容	行数 (估计)
`src/utls/gracefulShutdown.ts`	信号处理、分阶段关闭、故障安全	~300
`src/services/preventSleep.ts`	Mac caffeinate 管理器 (5min 自愈)	~100
`src/utls/cleanup.ts`	磁盘 GC (30 天轮换)	~300
`src/utls/cleanupRegistry.ts`	关闭钩子注册	~50
`src/utls/sequential.ts`	防竞态队列序列化	~50
`src/utls/abortController.ts`	子 AbortController 工厂	~100
`src/utls/combinedAbortSignal.ts`	超时 + 信号组合	~50
`src/utls/sleep.ts`	可中断睡眠	~30
`src/utls/timeouts.ts`	超时配置	~50

安全与权限

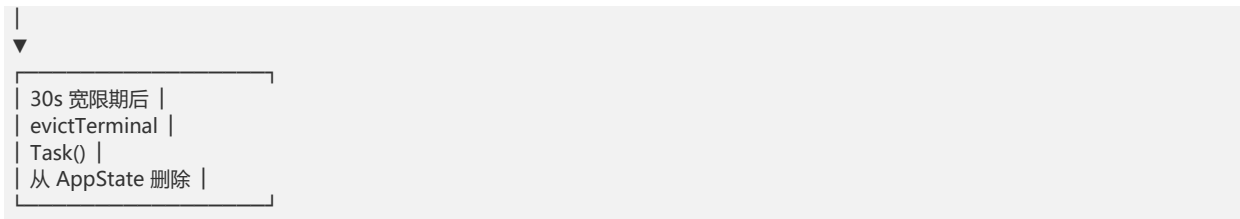
文件	内容	行数 (估计)
`src/hooks/toolPermission/`	权限状态机	~500
`src/utls/secureStorage/fallbackStorage.ts`	keychain → 纯文本回退	~100

文件	内容	行数 (估计)
`src/hooks/useCancelRequest.ts`	用户取消 (Escape/Ctrl+C)	~100
`src/utils/execFileNoThrow.ts`	永不抛出的 exec 包装	~50
`src/utils/process.ts`	EPIPE 处理	~50

附录：核心数据流图

子 Agent 完整生命周期





优雅关闭完整流程



- > 总结：Claude Code 的可靠性不是靠某一个"超级机制"实现的，而是靠多个子系统各司其职、层层兜底。每个子系统都遵循"先想清楚为什么失败，再决定怎么处理"的原则。这种设计哲学值得在构建长时间运行的 AI Agent 系统时参考。
- >
- > 核心原则：宁可稳健地失败，也不悄悄地挂起；宁可优雅地降级，也不粗暴地崩溃。